Entrega 2

Testes, estatísticas e demonstração do agente

QualityBridge - Agente de Testes de Software

Relatório do agente QualityBridge: arquitetura, integração, validação, testes e documentação.

Campo	Identificação
Título do projeto/ecossistema de agentes	QualityBridge - Agente de Testes de Software
Curso/Disciplina/Turma	Engenharia de Software; AEP - Agentes de Software para Projetos da Escola de TI; ESOFT 8S - Turma B.
Integrantes da equipe (6)	Thiago Poliseli Silva (RA 25362233-2); Pedro Toscano (RA 25362292-2); Pedro Paulo Barbosa Arantes (RA 25362249-2); Lucas de Freitas Bovo (RA 25362304-2); Gabriel Felipe Alexandre dos Santos (RA 25362250-2); Giovanne Leite (RA 25362248-2).
ODS escolhida	ODS 9 - Indústria, Inovação e Infraestrutura (Agente 1, conforme Entrega 1).
Link/repositório da implementação	https://github.com/giovanne7/QualityBridge
Data	13 de novembro de 2026

Nota de escopo. Esta entrega reúne os seis integrantes e avalia exclusivamente o agente QualityBridge. As conclusões se limitam às evidências desta avaliação.

1. Resumo Executivo

O QualityBridge executa testes existentes de projetos pytest, Playwright e Jest, normaliza os resultados e registra relatórios e histórico. Na revisão reproduzida, os 94 testes do repositório passaram em 6,71 s; os 12 cenários determinísticos de RCA corresponderam a categoria, commit e evidências esperadas, com média de 0,12 ms; a suíte rápida executada em Docker passou com 19 de 19 testes. A demonstração flaky produziu duas aprovações e duas falhas em quatro reexecuções, com três alternâncias e recomendação de quarentena. As evidências confirmam o funcionamento do agente próprio.

2. Ambiente e Metodologia de Teste

A suíte do repositório foi executada em container de validação com CPython 3.12.16, pytest 9.1.1, Git 2.47.3 e rede desativada; Ruff foi aplicado a src e tests. As demonstrações dos projetos pytest e Playwright foram executadas em Windows 11, Node.js 22 e Chromium 153.0.8010.12. A validação também usou os doze cenários RCA e quatro reexecuções de um projeto flaky. A imagem do agente foi executada em Docker desktop-linux, sem rede, na suíte quick.

As entradas incluíram a suíte de testes do repositório, doze cenários RCA JSON com causa conhecida, arquivos qualitybridge.yaml dos exemplos pytest e Playwright, e um projeto flaky repetido quatro vezes. Foram observados código de saída, estado dos casos, normalização, relatórios HTML/JSON/JUnit, histórico, hipóteses com evidências e alternância. A aceitação exigiu correspondência com os resultados esperados de cada cenário. A suíte inclui bordas como configuração e suíte inválidas, XML inseguro, ausência de testes, timeout, executável ausente e caminho de proposta fora do projeto. Os casos RCA são sintéticos e alinhados à heurística; 12/12 não estima precisão em falhas reais. Os tempos são medições locais, não benchmark controlado.

Critérios e ferramentas

Ferramenta/cenário	Critério observado
pytest e Ruff	Suíte completa sem falhas; análise estática sem apontamentos.
RCA determinístico	Categoria, commit e fundamentação de acordo com os 12 cenários conhecidos.
Projetos exemplo	Resultados por runner e relatórios correspondem ao que os casos de demonstração esperam.
Docker	Imagem construída; execução isolada da suíte quick concluída com status PASSED.

3. Testes, Estatísticas e Demonstração do QualityBridge

Agente 1 - Agente de Testes de Software (QualityBridge)

Confirmação da base tecnológica e do modelo de interação (Entrega 1)

A Entrega 1 especifica arquitetura própria determinística, sem LLM, e operação independente pela CLI. O código mantém o fluxo principal determinístico, mas também oferece LLM opcional para complementar o RCA. Essa opção não foi acionada nem medida. Há divergência entre documentos: o plano atribui integração LLM ao integrante 3, enquanto a Entrega 1 a exclui do Agente 1; portanto, a opção no código é uma extensão da especificação da Entrega 1, não evidência de que o requisito sem LLM foi seguido integralmente. A execução permanece independente e não orquestra agentes externos.

Regras, entradas e formatos

Entrada: projeto existente com testes e arquivo qualitybridge.yaml, contendo runner, caminho e suíte selecionada (full ou nomeada, como quick). A CLI valida a configuração e chama o Controller; Factory Method e adapters formam comandos específicos para pytest, Playwright ou Jest. Os resultados JUnit de pytest/Playwright e JSON do Jest são convertidos para RunResult e TestCase, preservando status, duração, saída e evidências. Regras: executar somente testes existentes e selecionados; não criar testes nem alterar o código do projeto; manter execução sob demanda pela CLI. Saídas: resumo de CLI, HTML, JSON, JUnit XML, log e histórico SQLite. O RCA correlaciona falhas, stack traces, Git e histórico; a opção LLM é adicional e não foi acionada nesta avaliação.

Casos de teste executados

Nº	Cenário testado	Resultado esperado	Resultado obtido
1	Suíte automatizada do repositório (pytest -q).	Todos os testes sem regressões.	94 aprovados; 6,71 s.
2	RCA em cenários com causas conhecidas.	Categoria, commit e evidências conforme o cenário.	12/12 em categoria, commit e evidência; média 0,12 ms/caso.
3	Exemplo pytest, full e quick.	Full: 4 aprovados e 1 ignorado; quick: 1 aprovado.	Resultados esperados observados nas duas suítes.
4	Exemplo Playwright com Chromium, full e quick.	Full: 2 aprovados; quick: 1 aprovado.	Resultados esperados observados nas duas suítes.
5	Demonstração flaky com quatro reexecuções.	Alternância identificada e suspeita sinalizada.	2 aprovações, 2 falhas, 3 alternâncias; suspeito com quarentena recomendada.

Métricas e estatísticas coletadas

Métrica	Valor obtido	Meta/critério de aceitação
Correspondência nos cenários RCA	100% (12/12)	12 cenários determinísticos concordantes; limitado aos casos conhecidos.
Tempo médio por cenário RCA	0,12 ms/caso	Medição local; sem meta numérica definida na Entrega 1.
Suíte do repositório / Docker quick	94 aprovados; 6,71 s. Docker quick: 19/19 aprovados.	Sem falhas nos testes executados; Docker validado em execução separada.

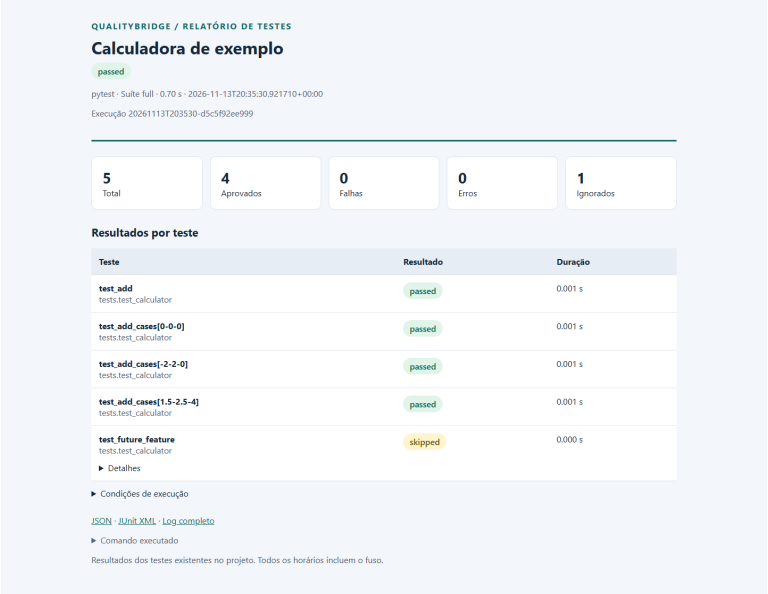
Gráfico dos resultados



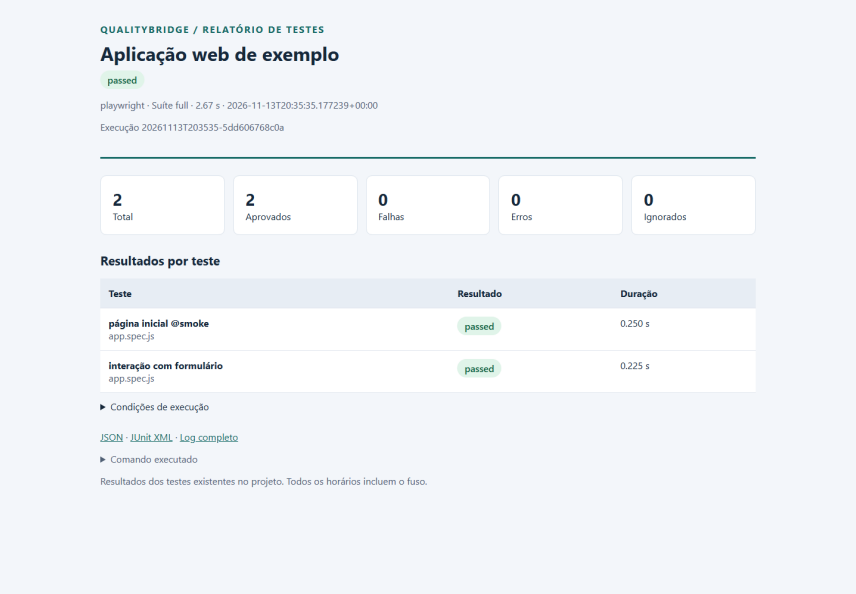
Execução observada: pytest -q; 94 aprovados, 0 falhas, 0 erros; 6,71 s.

Evidência antes/depois

A revisão corrigiu o rótulo de condições de execução no template HTML. As capturas mostram os relatórios finais; não foi preservada captura anterior para comparação visual.



Relatório HTML do exemplo pytest: quatro aprovados e um ignorado; execução 20261113T203530-d5c5f92ee999.



Relatório HTML do exemplo Playwright: dois aprovados; execução 20261113T203535-5dd606768c0a.

Análise crítica do agente

O fluxo em camadas, o Controller, a seleção Strategy (suíte completa ou nomeada), o Factory Method e os adapters estão presentes no código e correspondem à arquitetura descrita na Entrega 1. Os contratos RunResult/TestCase e as saídas CLI/HTML/JSON/JUnit refletem a separação de domínio e apresentação prevista pelo MVC adaptado. As demonstrações ponta a ponta cobriram pytest e Playwright; o adapter/parser Jest foi exercitado na suíte automatizada, mas não teve demonstração ponta a ponta nesta revisão. O fluxo principal permanece determinístico e sob demanda. O LLM opcional no RCA é extensão do plano, mas diverge da Entrega 1, que exige base sem LLM; não foi ativado nem avaliado, portanto sua qualidade não pode ser afirmada. Mantê-lo opcional preserva o caminho determinístico, mas a conformidade integral ao requisito sem LLM depende de decisão da equipe. Os 12 cenários sintéticos, definidos junto à heurística, demonstram consistência com casos previstos, não precisão externa.

4. Consolidação Estatística do QualityBridge

Execução/cenário	Entrada e regra observada	Resultado medido
Suíte do repositório	94 testes do QualityBridge; coletar e normalizar todos os casos.	94 aprovados, 0 falhas; 6,71 s.
Validação RCA	12 cenários JSON com categoria e commit esperados; cada hipótese deve citar evidência existente.	12/12 categoria, commit e evidência; 0,12 ms/caso em média.
pytest full e quick	Projeto exemplo + qualitybridge.yaml; full executa a suíte, quick seleciona 1 caso.	Full: 4 aprovados, 1 ignorado. Quick: 1 aprovado. IDs: 20261113T203530-d5c5f92ee999; 20261113T203533-af5902f6df66.
Playwright full e quick	Projeto web + YAML, Chromium real; quick seleciona 1 caso.	Full: 2 aprovados. Quick: 1 aprovado. IDs: 20261113T203535-5dd606768c0a; 20261113T203552-aa6caaaad065.
Demonstração flaky	Projeto com caso intermitente, quatro reexecuções controladas; medir falhas e alternâncias.	2 aprovados, 2 falhas, 3 alternâncias; marcado suspeito, quarentena recomendada.
Docker quick	Imagem qualitybridge:local, rede desativada, suíte rápida.	19 aprovados, 0 falhas; ID 20261113T204511-eb0abba2bb7b.
Análise estática	Ruff em src e tests.	All checks passed.

Leitura dos resultados: as 94 aprovações e a análise estática confirmam a suíte atual do agente. Os projetos pytest e Playwright verificam dois formatos de runner; o cenário flaky confirma detecção de alternância. O RCA atingiu os casos previstos porque o conjunto é sintético e alinhado às regras; não mede generalização. O teste que integra com Git passou no container com Git instalado.

5. Demonstração dos Resultados

A execução reproduzível em Docker foi `qualitybridge run --docker --suite quick`: 19 testes aprovados, 0 falhas, 0 erros; identificador 20261113T204511-eb0abba2bb7b. A imagem `qualitybridge:local` foi construída antes dessa execução. Os relatórios dos exemplos pytest e Playwright estão ilustrados na seção do Agente 1; o histórico, os relatórios JSON/JUnit e os logs ficam nas pastas `.qualitybridge/runs/` correspondentes, incluindo `.qualitybridge/runs/20261113T204511-eb0abba2bb7b/` para Docker. O roteiro de aceitação e as evidências são referenciados ao final.

6. Retrospectiva de Gestão do Projeto

Aderência à metodologia planejada

A implementação segue a arquitetura em camadas, os contratos compartilhados e os adaptadores previstos para o QualityBridge. A execução ocorre sob demanda pela CLI e não há orquestração de agentes externos. A Entrega 1 exclui LLM do Agente 1, enquanto o plano do grupo prevê LLM no RCA; o código inclui essa opção, mas ela não foi avaliada nesta revisão. CI permanece como etapa futura na Entrega 1. A revisão combinou testes automatizados, cenários conhecidos e exemplos de runners.

O que funcionou bem

Contratos comuns permitiram conectar diferentes frameworks; testes automatizados cobriram erros e integrações; relatórios e IDs de execução preservaram evidências; a execução Docker forneceu isolamento reproduzível; e o registro de resultados ajudou a separar fatos observados de limitações.

O que não funcionou / dificuldades enfrentadas

A equipe de seis integrantes foi dividida em duas frentes de três para organizar o trabalho; esta entrega consolida a avaliação do único agente QualityBridge. A principal divergência de escopo foi a previsão de LLM no RCA pelo plano, em contraste com a especificação sem LLM da Entrega 1; a avaliação usou apenas a heurística. O Docker Engine ficou inicialmente inacessível; depois de iniciar desktop-linux, a imagem foi construída e a suíte quick passou. A Entrega 1 não define metas quantitativas para todas as métricas, o que limitou comparações com um alvo pré-estabelecido.

Lições aprendidas

Na próxima revisão, registrar responsáveis por atividade e checkpoints do grupo, definir critérios quantitativos antes dos testes e preservar evidências antes/depois de correções. Para o RCA, ampliar a validação com falhas reais e manter os cenários sintéticos como teste de regressão.

7. Conclusão

A validação demonstrou que o QualityBridge executa e normaliza testes de múltiplos runners, registra histórico e fornece análises de RCA e intermitência com evidências. Nesta revisão, 94 testes passaram, Ruff não encontrou problemas e os 12 cenários RCA sintéticos corresponderam aos resultados esperados. A avaliação confirma comportamento nos casos executados; não demonstra precisão em falhas reais nem qualidade do complemento LLM, que não foi ativado. A relação com a ODS 9 é indireta: automatizar verificações pode reduzir retrabalho e contribuir para sistemas mais confiáveis.

8. Referências Utilizadas

• QUALITYBRIDGE. Repositório do projeto. Disponível em: https://github.com/giovanne7/QualityBridge. Acesso em: 13 nov. 2026.

• QUALITYBRIDGE. README, arquitetura, roteiro de aceitação, validação e RCA. Documentação técnica no repositório do projeto. Acesso em: 13 nov. 2026.

• NAÇÕES UNIDAS. Goal 9: Industry, innovation and infrastructure. Disponível em: https://sdgs.un.org/goals/goal9. Acesso em: 9 set. 2026.

• FOWLER, Martin. Presentation Domain Data Layering. 2015. Disponível em: https://martinfowler.com/bliki/PresentationDomainDataLayering.html. Acesso em: 9 set. 2026.

• GAMMA, Erich et al. Design Patterns: Elements of Reusable Object-Oriented Software. Reading: Addison-Wesley, 1995.

• LARMAN, Craig. Applying UML and Patterns. 3. ed. Upper Saddle River: Prentice Hall, 2005.

• PYTEST. Managing pytest’s output. Disponível em: https://docs.pytest.org/en/stable/how-to/output.html. Acesso em: 13 nov. 2026.

• MICROSOFT. Playwright Test: Command line. Disponível em: https://playwright.dev/docs/test-cli. Acesso em: 13 nov. 2026.

• QUALITYBRIDGE. AEP — Agentes de Software para Projetos da Escola de TI. Entrega 1 — Especificação de Agentes Selecionados. Documento PDF fornecido pela equipe.

• PLANO DE DIVISÃO DE ATIVIDADES. Agente de IA para Code Review e Testes. Documento PDF fornecido pela equipe.